

LC 3027 - Find the Number of Ways to Place People II

Google

Uber

Q 282

Problem Insights & Approach

Just like LC 3025, we need to find 2 pairs such that Aadya at top left corner and Bhodh at bottom right corner, and no one in between them.

A clever optimization to 3025: Sort all points in a way that x is ascending and if x is same then y is descending.

Doing this would mean x automatically satisfies condition $x_A \leq x_B$. Now only need to worry about y .

This effectively removes the third loop. We count the answer ($ans++$) only if $y >$ last accepted y .

Example Walkthrough

Now suppose Aadya is at $(2, 8)$. Now she scans right for Bhodh's position:

Point Checked	Condition	Result
$(3, 9)$	$9 > 8$	Cant be
$(4, 7)$	Valid lower y	Works, count it
$(5, 1)$	$y >$ previous accepted y	Any later point must satisfy this.

*Note from diagram: Points are not valid as then 1 and 2 which both are bigger than it would come inside.

Complexity

TC = $O(n^2)$

SC = $O(1)$

Implementation

```
class Solution {
public:
    int numberOfPairs(vector<vector<int>>& points) {
        int n=points.size();
        int ans=0;

        sort(points.begin(),points.end(),[](vector<int>& a,vector<int>&
b){
            if(a[0]==b[0]){
                return a[1]>b[1];
            }
            return a[0]<b[0];           //sort by increasing order
of x , and if x same then descending order of y
        });

        //now fix position of aadya (can i fix her? TwT)
        for(int i=0;i<n;i++){

            int lasty=INT_MIN;

            for(int j=i+1;j<n;j++){
                if(points[j][1]>points[i][1]){
                    continue;
                }

                if(points[j][1]>lasty){
                    ans++;
                    lasty=max(lasty,points[j][1]);
                }
            }
        }

        return ans;
    }
};
```

```
}  
};
```